

ADVANCED SQL

Capstone Project

Million-Row Retail Sales Database Analysis

PostgreSQL 16	pgAdmin 4	Indexing
Window Functions	CTEs	Business Intelligence

Instructor: Rajdeep Ray

GRRAS Solutions, Jaipur · grras.com

PostgreSQL 16 · pgAdmin 4 · Builds on Classes 1, 2 & 3

Project Objective

In this capstone project, you will work with a real-world retail sales database containing **over 1 million transaction records**. Your goal is to write progressively complex SQL queries — from simple aggregations to advanced window functions and CTEs — to extract meaningful business intelligence from raw data.

What You Will Practice

Database Schema Design

Design and deploy a normalized multi-table schema from scratch in PostgreSQL.

Performance Optimization

Use B-Tree indexes to reduce query execution time by up to 600%.

Aggregation & Filtering

Write GROUP BY, HAVING, and multi-table JOIN queries for business reporting.

Date & Time Analysis

Work with date arithmetic, EXTRACT, TO_CHAR, and interval filtering.

Window Functions

Use RANK, LAG, and PARTITION BY to analyze trends across rows.

CTEs & Subqueries

Build multi-step analytical pipelines using chained CTEs.

✓ Your Approach

Break every complex query into steps. Write and test each step separately in pgAdmin 4. Once each part works, combine them. Read error messages carefully — PostgreSQL is very descriptive.

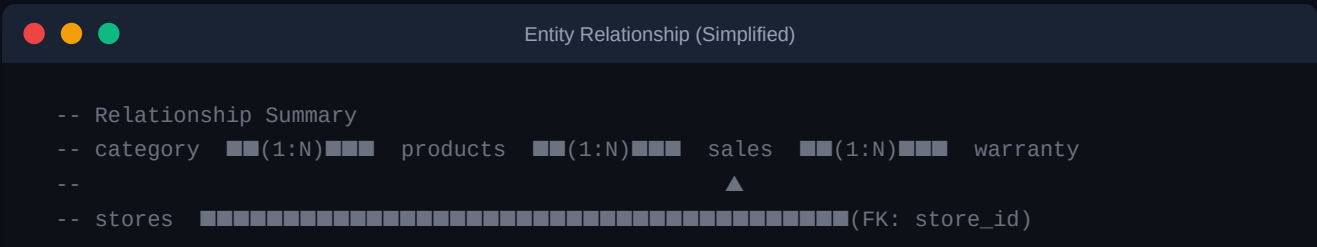
The Dataset

The project uses a retail sales database called **apple_db** built in PostgreSQL. It models the operations of a global electronics retailer with stores across multiple countries. The database has **5 tables** and contains over **1 million rows** of transactional data.

stores	store_id, store_name, city, country	73+ global retail locations
category	category_id, category_name	10 product categories
products	product_id, product_name, category_id, launch_date, price	64+ items with retail prices in USD
sales	sale_id, sale_date, store_id, product_id, quantity	Core fact table with 1M+ transaction rows
warranty	claim_id, claim_date, sale_id, repair_status	30,000 warranty claim records

How the Tables Relate

The **sales** table is the central *fact table*. It links to **stores** and **products** via foreign keys. The **products** table links to **category**. The **warranty** table links back to individual sale records through **sale_id**.



Questions We Will Answer

This project covers **10 business questions** of increasing difficulty. Together they cover the complete analytical cycle: from basic aggregation to advanced window functions.

#	Business Question	Difficulty
Q1	Count the number of stores in each country	Medium
Q2	Total units sold by each store	Medium
Q3	Total transactions in December 2023	Medium
Q4	Stores that never received a warranty claim	Medium
Q5	Percentage of warranty claims marked as Warranty Void	Medium
Q6	Store with highest unit sales in the last year	Medium

#	Business Question	Difficulty
Q7	Count unique products sold in the last year	Medium
Q8	Average price per product category	Medium
Q9	Warranty claims filed in 2020	Medium
Q10	Best-selling weekday per store (Window Function)	Hard

Why Indexes Matter

With over 1 million rows in the sales table, every query you write does a **sequential scan** by default — the engine reads every single row from disk. Creating indexes on the foreign key columns transforms these into fast **Bitmap Index Scans**.

Scan Type	product_id lookup	store_id lookup
Sequential Scan (no index)	~64 ms	~58 ms
Bitmap Index Scan (with index)	~4–10 ms	~2 ms
Improvement	6–16×	29×

Creating the Indexes

```
-- Run these once after loading the dataset
-- PostgreSQL automatically uses them for any query filtering on these columns

CREATE INDEX idx_sales_product_id
  ON sales(product_id);

CREATE INDEX idx_sales_store_id
  ON sales(store_id);

CREATE INDEX idx_sales_sale_date
  ON sales(sale_date);

-- Verify indexes were created
SELECT indexname, indexdef
FROM pg_indexes
WHERE tablename = 'sales';
```

■ Run EXPLAIN ANALYZE

Before and after creating indexes, run `EXPLAIN ANALYZE SELECT ...` on any query to see the execution plan. Look for "Seq Scan" (slow) vs "Bitmap Index Scan" (fast) in the output.

How Each Question Is Structured

Every question follows the same four-part format: **(1)** a plain-English objective, **(2)** a numbered breakdown of the mini-steps, **(3)** intermediate code blocks you can run individually to test each step, and **(4)** the complete final query ready to copy and run.

Q1 Number of Stores Per Country

Medium

Objective

Count how many retail store locations exist in each country. The result should be sorted so the country with the most stores appears first.

Step-by-Step Approach

Step	Action
1	Identify the source table — stores contains one row per location.
2	Choose the grouping column — country is what we want to group by.
3	Choose the aggregate — COUNT(store_id) gives the store count per group.
4	Sort the output with ORDER BY total_stores DESC.

Intermediate Steps



Q1 — Exploration Steps

```
-- Step 1: Confirm the column exists and check sample data
SELECT DISTINCT country
FROM stores
ORDER BY country;

-- Step 2: Add COUNT before applying GROUP BY
SELECT country, COUNT(store_id)
FROM stores
GROUP BY country;
```

Final Complete Query



Q1 — Final Query

```
SELECT
    country,
    COUNT(store_id) AS total_stores
FROM stores
GROUP BY country
ORDER BY total_stores DESC;
```

Things to Look Out For

- Every column in `SELECT` must either be in `GROUP BY` or wrapped in an aggregate function.
- `COUNT(store_id)` and `COUNT(*)` give the same result here — `store_id` has no `NULL`s. `COUNT(*)` is fine.
- `GROUP BY` runs after `WHERE` but before `HAVING` — no `HAVING` needed here since we want all countries.
- Aliases defined in `SELECT` (e.g. `total_stores`) cannot be used inside `GROUP BY` — use the original column name.

Q2 Total Units Sold By Each Store

Medium

Objective

Find the total number of product units sold by every store, ranked from highest to lowest. This requires joining the sales fact table with the stores dimension table to get human-readable store names.

Step-by-Step Approach

Step	Action
1	The quantity column lives in sales. The store_name lives in stores.
2	Join the two tables on the shared key: sales.store_id = stores.store_id.
3	Use SUM(quantity) — not COUNT(*) — to get actual units moved.
4	Group by store_name and sort descending.

Intermediate Steps

Q2 — Exploration Steps

```
-- Step 1: Check what the raw join looks like before aggregating
SELECT s.store_id, st.store_name, s.quantity
FROM sales s
INNER JOIN stores st ON s.store_id = st.store_id
LIMIT 10;

-- Step 2: Add the SUM and GROUP BY
SELECT st.store_name, SUM(s.quantity)
FROM sales s
INNER JOIN stores st ON s.store_id = st.store_id
GROUP BY st.store_name;
```

Final Complete Query

Q2 — Final Query

```
SELECT
    st.store_name,
    SUM(s.quantity) AS total_units_sold
FROM sales s
INNER JOIN stores st
    ON s.store_id = st.store_id
GROUP BY st.store_name
ORDER BY total_units_sold DESC;
```

Things to Look Out For

- Critical trap: `COUNT(*)` counts the number of invoice rows, not the number of units. A single sale row can have quantity = 10. Always use `SUM(quantity)` for volume.
- `INNER JOIN` is correct here — we only want stores that actually made sales. A `LEFT JOIN` would include stores with zero sales showing `NULL`.
- Use table aliases (`s` for sales, `st` for stores) when joining — it keeps long queries readable.
- The grouping column must be `st.store_name`, not `s.store_id`, since we want names in the output.

Q3

Total Transactions in December 2023

Medium

Objective

Count the total number of sales transactions that occurred exclusively during the month of December 2023. Two approaches are shown — one readable, one index-optimized.

Step-by-Step Approach

Step	Action
1	The sale_date column in sales holds our dates (stored as DATE type).
2	Option A: Use TO_CHAR to convert the date to 'YYYY-MM' string format and compare.
3	Option B: Use BETWEEN with explicit date strings — this is index-friendly.
4	Wrap sale_id in COUNT() to count distinct transactions.

Intermediate Steps



Q3 — Readable Version (Option A)

```
-- Option A: Readable — using TO_CHAR (avoids index)
SELECT COUNT(sale_id) AS total_sales
FROM sales
WHERE TO_CHAR(sale_date, 'YYYY-MM') = '2023-12';
```

Final Complete Query



Q3 — Final Query (Index-Optimized)

```
-- Option B: Index-Optimized (preferred for large tables)
SELECT
    COUNT(sale_id) AS total_sales
FROM sales
WHERE sale_date BETWEEN '2023-12-01' AND '2023-12-31';
```

Things to Look Out For

- Wrapping a column inside a function like TO_CHAR(sale_date, ...) prevents PostgreSQL from using the index on sale_date. On a 1M-row table this matters.
- BETWEEN '2023-12-01' AND '2023-12-31' is the preferred approach in production — it uses the index.
- COUNT(sale_id) and COUNT(*) both work here. COUNT(*) is marginally faster since it does not check for NULLs.
- If you need to group by month across multiple years (e.g. monthly revenue report), TO_CHAR is fine — use GROUP BY TO_CHAR(sale_date, 'YYYY-MM').

Q4

Stores That Never Received a Warranty Claim

Medium

Objective

Find how many stores have never had a customer file a warranty claim. This requires identifying which store IDs appear in warranty records and then excluding them.

Step-by-Step Approach

Step	Action
1	The warranty table does not have store_id directly — we must go through sales.
2	Inner subquery: join warranty to sales to get all store_ids that have claims.
3	Use DISTINCT to avoid duplicate store_ids in the subquery result.
4	Outer query: use NOT IN on stores to exclude those store_ids.
5	Wrap in COUNT(*) to get the final number.

Intermediate Steps



Q4 — Exploration Steps

```
-- Step 1: Find all store_ids that DO have warranty claims
SELECT DISTINCT s.store_id
FROM warranty w
INNER JOIN sales s ON w.sale_id = s.sale_id
WHERE s.store_id IS NOT NULL;

-- Step 2: Test the NOT IN logic before wrapping in COUNT
SELECT store_id, store_name
FROM stores
WHERE store_id NOT IN (
    SELECT DISTINCT s.store_id
    FROM warranty w
    INNER JOIN sales s ON w.sale_id = s.sale_id
    WHERE s.store_id IS NOT NULL
);
```

Final Complete Query

```
SELECT
    COUNT(*) AS stores_with_no_claims
FROM stores
WHERE store_id NOT IN (
    SELECT DISTINCT s.store_id
    FROM warranty w
    INNER JOIN sales s
        ON w.sale_id = s.sale_id
    WHERE s.store_id IS NOT NULL
);
```

Things to Look Out For

- warranty links to sales via sale_id — not directly to stores. You must join warranty to sales first to get store_id.
- Always add WHERE s.store_id IS NOT NULL inside the subquery when using NOT IN. If even one NULL slips through, NOT IN returns zero rows for the entire outer query.
- Alternatively use NOT EXISTS for safer null handling — shown in the intermediate section.
- To list the actual store names (not just a count), remove COUNT(*) and SELECT store_name instead.

Q5

Percentage of Claims Marked as Warranty Void

Medium

Objective

Calculate what percentage of all warranty claims carry the status "Warranty Void" — meaning the claim was rejected, typically due to tampering or water damage.

Step-by-Step Approach

Step	Action
1	The warranty table has repair_status. Filter for 'Warranty Void'.
2	Use a CASE WHEN inside COUNT() to count only the matching rows.
3	Divide by COUNT(*) (total claims) and multiply by 100.0 (not 100) to get percentage.
4	Wrap in ROUND(..., 2) for a clean two-decimal output.

Intermediate Steps



Q5 — Exploration Steps

```
-- Step 1: Count total claims vs void claims separately
SELECT
    COUNT(*)                                AS total_claims,
    COUNT(CASE WHEN repair_status = 'Warranty Void'
           THEN 1 END)                       AS void_count
FROM warranty;

-- Step 2: Show integer division problem
SELECT 3 / 10;      -- returns 0 (wrong!)
SELECT 3 * 100.0 / 10; -- returns 30.0 (correct)
```

Final Complete Query



Q5 — Final Query

```
SELECT
    ROUND(
        COUNT(CASE WHEN repair_status = 'Warranty Void'
                 THEN 1 END) * 100.0
        / COUNT(*),
        2
    ) AS void_percentage
FROM warranty;
```

Things to Look Out For

- Integer division trap: in PostgreSQL, `integer / integer = integer`. $3 / 10 = 0$, not 0.3. Multiplying by 100.0 (a float) forces the engine to return a decimal result.
- `CASE WHEN` inside `COUNT()` is conditional aggregation — a single-pass technique to count rows matching a condition without a `WHERE` clause subquery.
- An alternative approach: two separate `SELECT`s — one for void count, one for total — wrapped in a CTE. Both are correct; the single-pass version is more efficient.
- `NULLIF(COUNT(*), 0)` can protect against division by zero if the table is empty.

Q6

Store With Highest Sales in the Last Year

Medium

Objective

Find the single store that sold the most units over the trailing 12-month period. The time window should be dynamic — it should always be relative to the current date, not a hardcoded range.

Step-by-Step Approach

Step	Action
1	Join sales to stores to get store names.
2	Filter using CURRENT_DATE - INTERVAL '1 year' for a dynamic date boundary.
3	SUM(quantity) per store and ORDER BY descending.
4	Apply LIMIT 1 to return only the top performer.

Intermediate Steps



Q6 — Exploration Steps

```
-- Step 1: Confirm the date filter works correctly
SELECT MIN(sale_date), MAX(sale_date)
FROM sales
WHERE sale_date >= CURRENT_DATE - INTERVAL '1 year';

-- Step 2: Preview the join before aggregating
SELECT st.store_name, s.quantity
FROM sales s
INNER JOIN stores st ON s.store_id = st.store_id
WHERE s.sale_date >= CURRENT_DATE - INTERVAL '1 year'
LIMIT 5;
```

Final Complete Query



Q6 — Final Query

```
SELECT
    st.store_name,
    SUM(s.quantity) AS total_units
FROM sales s
INNER JOIN stores st
    ON s.store_id = st.store_id
WHERE s.sale_date >= CURRENT_DATE - INTERVAL '1 year'
GROUP BY st.store_name
ORDER BY total_units DESC
LIMIT 1;
```


Things to Look Out For

- Never hardcode dates like `WHERE sale_date >= '2024-01-01'`. Use `CURRENT_DATE - INTERVAL '1 year'` so the query stays accurate as time passes.
- `INTERVAL '1 year'` is cleaner than `INTERVAL '365 days'` — it handles leap years correctly.
- `LIMIT 1` is index-efficient when combined with `ORDER BY` on an indexed column.
- To handle ties (two stores with the exact same total), use `RANK() OVER(ORDER BY total_units DESC)` instead of `LIMIT`.

Q7

Count of Unique Products Sold in the Last Year

Medium

Objective

Determine how many distinct product SKUs were sold over the trailing 12 months. This measures catalog diversity — how many different items are actively moving off shelves.

Step-by-Step Approach

Step	Action
1	Filter sales to the last 12 months using <code>CURRENT_DATE - INTERVAL</code> .
2	Use <code>COUNT(DISTINCT product_id)</code> to deduplicate product IDs.
3	No JOIN needed — <code>product_id</code> already exists in the sales table.

Intermediate Steps

Q7 — Exploration Steps

```
-- Step 1: Check raw product_ids inside the time window
SELECT product_id
FROM sales
WHERE sale_date >= CURRENT_DATE - INTERVAL '1 year'
LIMIT 20;

-- Step 2: Count without DISTINCT to see the difference
SELECT COUNT(product_id) AS all_rows
FROM sales
WHERE sale_date >= CURRENT_DATE - INTERVAL '1 year';
```

Final Complete Query

Q7 — Final Query

```
SELECT
    COUNT(DISTINCT product_id) AS unique_skus_sold
FROM sales
WHERE sale_date >= CURRENT_DATE - INTERVAL '1 year';
```

Things to Look Out For

- `COUNT(DISTINCT column)` deduplicates automatically. Without `DISTINCT`, you would count every transaction row, not unique products.
- This is different from Q2 — we do not care about volume (units) here, only breadth (how many different products).
- To see which products were NOT sold in the last year, use `NOT IN` or `NOT EXISTS` against the products table.
- On very large datasets, `COUNT(DISTINCT)` can be slow. For approximations, PostgreSQL supports HyperLogLog via extensions.

Q8

Average Product Price Per Category

Medium

Objective

Calculate the average retail price of products in each category, sorted from most expensive to least expensive. This requires joining products to category to get readable category names.

Step-by-Step Approach

Step	Action
1	Product prices are in the products table (price column).
2	Category names are in the category table (category_name).
3	Join the two tables on category_id.
4	GROUP BY category_name and apply AVG(price), wrapped in ROUND.

Intermediate Steps



Q8 — Exploration Steps

```
-- Step 1: Preview the join
SELECT p.product_name, p.price, c.category_name
FROM products p
INNER JOIN category c ON p.category_id = c.category_id
LIMIT 10;

-- Step 2: Add aggregation
SELECT c.category_name, AVG(p.price)
FROM products p
INNER JOIN category c ON p.category_id = c.category_id
GROUP BY c.category_name;
```

Final Complete Query



Q8 — Final Query

```
SELECT
    c.category_name,
    ROUND(AVG(p.price), 2) AS avg_retail_price
FROM products p
INNER JOIN category c
    ON p.category_id = c.category_id
GROUP BY c.category_name
ORDER BY avg_retail_price DESC;
```

Things to Look Out For

- Group by the descriptive name (`category_name`), not the numeric ID (`category_id`). This makes the output readable without a second lookup.
- `ROUND(AVG(price), 2)` gives a clean two-decimal result. Without `ROUND`, you may get 15+ decimal places.
- If some products have no category (orphan rows), a `LEFT JOIN` would include them as a `NULL` group. `INNER JOIN` silently drops them.
- You could also use `category_id` in `GROUP BY` and `JOIN` afterwards — both produce the same result, but grouping by name is cleaner.

Q9

Total Warranty Claims Filed in 2020

Medium

Objective

Count the number of warranty claims that were submitted specifically during the calendar year 2020. Practice using EXTRACT to pull the year component from a date column.

Step-by-Step Approach

Step	Action
1	The claim_date column in warranty holds submission dates.
2	Use EXTRACT(YEAR FROM claim_date) to isolate the year component.
3	Compare with = 2020 (integer, not a string).
4	Wrap claim_id in COUNT() for the final count.

Intermediate Steps



Q9 — Exploration Steps

```
-- Step 1: Verify EXTRACT output on a few rows
SELECT
    claim_date,
    EXTRACT(YEAR FROM claim_date) AS claim_year
FROM warranty
LIMIT 10;

-- Step 2: Index-friendly alternative using BETWEEN
SELECT COUNT(claim_id)
FROM warranty
WHERE claim_date BETWEEN '2020-01-01' AND '2020-12-31';
```

Final Complete Query



Q9 — Final Query

```
SELECT
    COUNT(claim_id) AS total_claims_2020
FROM warranty
WHERE EXTRACT(YEAR FROM claim_date) = 2020;
```

Things to Look Out For

- `EXTRACT` returns a numeric value, so compare with `= 2020`, not `= '2020'`. Comparing with a string works in some databases but is bad practice.
- `EXTRACT(YEAR FROM date)` vs `TO_CHAR(date, 'YYYY')`: `EXTRACT` gives a number, `TO_CHAR` gives a string. For year filtering, `EXTRACT` is cleaner.
- Like `TO_CHAR`, wrapping a column in `EXTRACT()` prevents index usage. For high-performance filtering, prefer `BETWEEN '2020-01-01' AND '2020-12-31'`.
- `DATE_PART('year', claim_date)` is functionally identical to `EXTRACT` — use `EXTRACT` for SQL-standard portability.

Q10

Best-Selling Weekday Per Store

Hard

Objective

For every individual store, find which day of the week generates the highest total sales volume. This is a classic "Top-N per Group" problem solved using a CTE combined with a window function.

Step-by-Step Approach

Step	Action
1	Aggregate total units sold per (store_id, weekday) combination.
2	Use TO_CHAR(sale_date, 'Day') to get the weekday name from each date.
3	Apply RANK() OVER(PARTITION BY store_id ORDER BY SUM(quantity) DESC) to rank weekdays within each store.
4	Wrap everything in a CTE (WITH clause).
5	In the outer query, filter WHERE sales_rank = 1 to return only the top day per store.

Intermediate Steps

Q10 — Intermediate Steps (test each part separately)

```
-- Step 1: Aggregate sales by store_id + weekday
SELECT
    store_id,
    TRIM(TO_CHAR(sale_date, 'Day')) AS weekday,
    SUM(quantity) AS total_units
FROM sales
GROUP BY store_id, TRIM(TO_CHAR(sale_date, 'Day'))
ORDER BY store_id, total_units DESC
LIMIT 20;

-- Step 2: Add the RANK window function (run this before wrapping in CTE)
SELECT
    store_id,
    TRIM(TO_CHAR(sale_date, 'Day')) AS weekday,
    SUM(quantity) AS total_units,
    RANK() OVER (
        PARTITION BY store_id
        ORDER BY SUM(quantity) DESC
    ) AS sales_rank
FROM sales
GROUP BY store_id, TRIM(TO_CHAR(sale_date, 'Day'))
ORDER BY store_id, sales_rank
LIMIT 30;
```

Final Complete Query

```

WITH daily_sales AS (
  SELECT
    s.store_id,
    st.store_name,
    TRIM(TO_CHAR(s.sale_date, 'Day')) AS weekday,
    SUM(s.quantity) AS total_units,
    RANK() OVER (
      PARTITION BY s.store_id
      ORDER BY SUM(s.quantity) DESC
    ) AS sales_rank
  FROM sales s
  INNER JOIN stores st
    ON s.store_id = st.store_id
  GROUP BY
    s.store_id,
    st.store_name,
    TRIM(TO_CHAR(s.sale_date, 'Day'))
)
SELECT
  store_name,
  weekday AS best_selling_day,
  total_units
FROM daily_sales
WHERE sales_rank = 1
ORDER BY store_name;

```

Things to Look Out For

- Window functions cannot appear in WHERE clauses. You must wrap the window query in a CTE and then filter on the rank in the outer SELECT.
- TO_CHAR(date, 'Day') returns a padded string like 'Monday '. Use TRIM(TO_CHAR(...)) if you want clean output.
- RANK() assigns the same rank to ties and skips the next number. If two weekdays have the same total for a store, both get rank 1. Use ROW_NUMBER() if you want exactly one row per store.
- The GROUP BY inside the CTE must include both store_id and weekday. The RANK() window function uses PARTITION BY store_id so it resets the ranking for each store.
- This CTE pattern — aggregate → rank → filter — is the standard "Top N per Group" template in SQL. Memorize it.

What We Accomplished

Over the course of this project you built a complete analytical pipeline on a million-row production-scale database. Here is a recap of every skill you applied:

Q#	Topic	Key Technique	Key Insight
Q1	Stores per Country	GROUP BY + COUNT	Aggregate rows into country-level summaries
Q2	Units Sold per Store	INNER JOIN + SUM(quantity)	Join fact and dimension tables; avoid COUNT(*) trap
Q3	December 2023 Transactions	WHERE + TO_CHAR / BETWEEN	Date filtering; index-aware alternatives
Q4	Stores with No Claims	NOT IN + Correlated Subquery	Negative membership filtering across tables
Q5	Warranty Void %	CASE WHEN inside COUNT + ROUND	Conditional aggregation; float division safeguard
Q6	Top Store Last Year	INTERVAL + ORDER BY + LIMIT 1	Dynamic time windows; avoid hardcoded dates
Q7	Unique SKUs Sold	COUNT(DISTINCT)	Deduplication within a date range
Q8	Avg Price per Category	INNER JOIN + AVG + ROUND	Joining dimension tables for readable output
Q9	Claims in 2020	EXTRACT(YEAR FROM date)	Numerical year extraction vs string casting
Q10	Best Weekday per Store	CTE + RANK() OVER(PARTITION BY)	Multi-step window function — the capstone pattern

Core Patterns to Remember

COUNT(*) vs SUM(quantity)

COUNT(*) counts rows (invoices). SUM(quantity) counts actual units. Always use SUM(quantity) for volume tracking.

Integer Division Trap

In PostgreSQL, $5/2 = 2$, not 2.5. Multiply by 100.0 (not 100) to force float division when computing percentages.

TO_CHAR vs BETWEEN for Dates

TO_CHAR(date, 'YYYY-MM') is readable but skips indexes. Use BETWEEN for high-performance date filtering on large tables.

NOT IN with NULLs

If a subquery used with NOT IN returns even one NULL, the entire outer query returns no rows. Use NOT EXISTS as a safer alternative.

Window Functions Need CTE

You cannot filter on a window function (e.g. `RANK()`) directly in `WHERE`. Wrap the query in a CTE, then filter in the outer `SELECT`.

Index Your Foreign Keys

Always `CREATE INDEX` on foreign key columns (`store_id`, `product_id`, `sale_date`) in large fact tables. This cuts scan time from 60ms to 2ms.

Next Steps

- Extend Q10 to find the worst-performing weekday per store.
- Write a Year-Over-Year growth query using `LAG()` partitioned by store.
- Build a Product Lifecycle Analysis segmenting sales by months since launch.
- Push your SQL scripts to GitHub with a documented README and ERD diagram.

Keep querying. Keep optimizing.

— Rajdeep Ray, GRRAS Solutions